Homework 1

Abby Kreuser

```
library(tidyverse)
library(lubridate)
```

Warm up

Given the vector below:

```
quiz_scores <- c(78, 85, 92, 67, 88, 95, 73, 84)
```

Write R code to:

1. Print the first score.
2. Print the last score.
3. Calculate the average score.
4. Create a new vector containing only scores greater than or equal to 80.
5. Count how many students scored below 80.

```
print(paste("1.", quiz_scores[1]))
```

```
## [1] "1. 78"
```

```
print(paste("2.", quiz_scores[length(quiz_scores)]))
```

```
## [1] "2. 84"
```

```
print(paste("3.", mean(quiz_scores)))
```

```
## [1] "3. 82.75"
```

```
below_80 <- quiz_scores[quiz_scores < 80]
below_80 <- paste(below_80)
print(paste("4. below 80:",paste(below_80, collapse = " ")))
```

```
## [1] "4. below 80: 78 67 73"
```

```
print(paste("5.", length(below_80)))
```

```
## [1] "5. 3"
```

Write a function called `letter_grade()` that takes a numeric score and returns:

Score Range	Grade
90 or above	A
80–89	B
70–79	C
60–69	D
Below 60	F

Example:

```
letter_grade(92)
```

Expected output:

```
[1] "A"
```

```
letter_grade<-function(score){
if(score >= 90){
    return("A")
  }
  if(score >= 80 & score <90){
    return("B")
  }
   if(score >= 70 & score <80){
    return("C")
  }
  if(score >= 60 & score <70){
    return("D")
  }
   if(score < 60){
    return("F")
  }
}
```

Then:

7. Create a data frame using the `quiz_scores` vector.
8. Create a new column named `Letter_Grade` by applying the `letter_grade()` function.

```
quiz_scores_df <- as.data.frame(quiz_scores)
Letter_Grade <- sapply(quiz_scores, letter_grade)
quiz_scores_df <- cbind(quiz_scores_df, Letter_Grade)
```

Add the following vector as a column named `HoursOfStudy`:

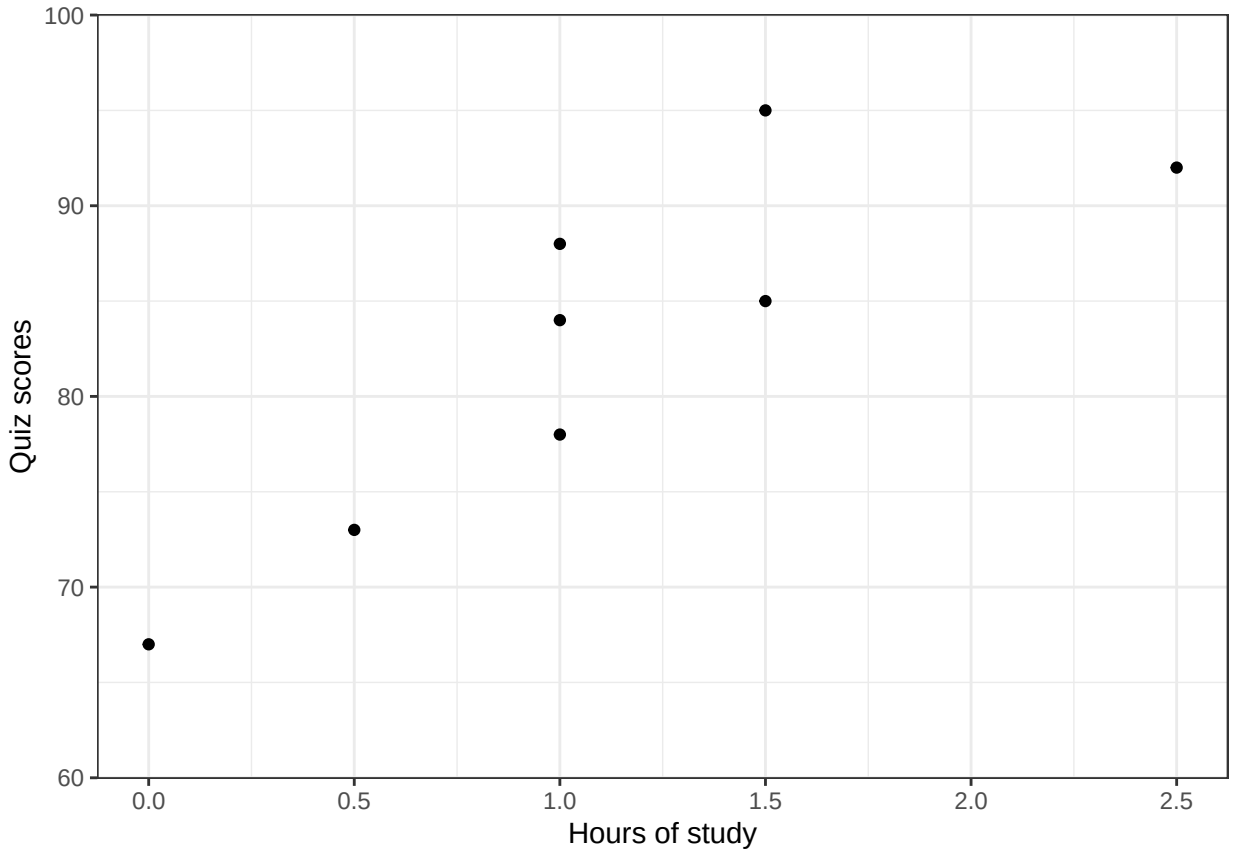
```
study_hours <- c("1", "1.5", "2.5", "0", "1", "1.5", "0.5", "1")
quiz_scores_df <- cbind(quiz_scores_df, study_hours )
colnames(quiz_scores_df)[3] <- "HoursOfStudy"
```

Once you create the column, do the following:

9. Change the column to numeric.
10. Create a plot using `HoursOfStudy` and `quiz_scores`.
11. Make the plot look polished and readable.

```
quiz_scores_df$HoursOfStudy <- as.numeric(quiz_scores_df$HoursOfStudy)

ggplot()+
  geom_point(data=quiz_scores_df, aes(x=HoursOfStudy, y=quiz_scores))+
  #ylim(c(60,100), )+
  scale_y_continuous(limits=c(60,100), expand=c(0.001))+
  ylab("Quiz scores")+
  xlab("Hours of study")+
  theme_bw()
```



Data Analysis

For this part, we will work with an actual dataset.

Load the following dataset:

https://raw.githubusercontent.com/dblaskey/ML_Course_Code/main/Data/weatherHistory.csv

```
weather_df <- read.csv("https://raw.githubusercontent.com/dblaskey/ML_Course_Code/main/Data/weatherHist
```

It is usually best practice to keep a raw copy of the original data unless the dataset is exceptionally large. Create a copy of the dataset.

```
raw_weather_df <- weather_df
```

Clean the column names of the data frame:

1. Remove spaces at the beginning and end of each column name.
2. Convert all column names to lowercase.
3. Replace any group of characters that is not a letter or number with an underscore.
4. Remove underscores from the beginning and end.

```
names(weather_df)
```

```
##  [1] "Formatted.Date"
##  [2] "Summary"
##  [3] "Precip.Type"
##  [4] "Temperature..C."
##  [5] "Apparent.Temperature..C."
##  [6] "Humidity"
```

```
##  [7] "Wind.Speed..km.h."
##  [8] "Wind.Bearing..degrees."
##  [9] "Visibility..km."
## [10] "Loud.Cover"
## [11] "Pressure..millibars."
## [12] "Daily.Summary"
```

```
names(weather_df) <- names(weather_df) |>
  trimws() |> #i dont see any spaces
  tolower() |>
  gsub("[^a-z0-9]+", "_", x = _) |>
  gsub("^_+|_+$", "", x = _)

names(weather_df)
```

```
##  [1] "formatted_date"         "summary"
##  [3] "precip_type"            "temperature_c"
##  [5] "apparent_temperature_c" "humidity"
##  [7] "wind_speed_km_h"        "wind_bearing_degrees"
##  [9] "visibility_km"          "loud_cover"
## [11] "pressure_millibars"     "daily_summary"
```

Now that you have cleaned the column names, it is time to understand and clean the dataset:

1. Notice there seems to be a typo in a column heading. Change `loud_cover` to `cloud_cover`.
2. Check for impossible data, such as precipitation, wind speed, visibility, cloud cover, or pressure below 0, and any variable outside reasonable earthly maximum and minimum values.
3. Give the count, mean, standard deviation, maximum, and minimum values for each numeric column.
4. Count the number of missing values in each column.
5. Create a new column that converts `formatted_date` to a date-time value.
6. Create new columns for hour, day, month, and year.

```
colnames(weather_df)[colnames(weather_df) == "loud_cover"] <- "cloud_cover"

summary(weather_df) #cloud cover all 0s
```

```
##  formatted_date       summary
##  Length:96453       Length:96453
##  Class :character   Class :character
##  Mode  :character   Mode  :character
##
##
##
##  precip_type        temperature_c
##  Length:96453       Min.   :-21.822
##  Class :character   1st Qu.:  4.689
##  Mode  :character   Median : 12.000
##                     Mean   : 11.933
##                     3rd Qu.: 18.839
##                     Max.   : 39.906
##  apparent_temperature_c    humidity
##  Min.   :-27.717        Min.   :0.0000
##  1st Qu.:  2.311        1st Qu.:0.6000
##  Median : 12.000        Median :0.7800
##  Mean   : 10.855        Mean   :0.7349
##  3rd Qu.: 18.839        3rd Qu.:0.8900
```

```
##  Max.   : 39.344        Max.   :1.0000
##  wind_speed_km_h  wind_bearing_degrees visibility_km
##  Min.   : 0.000   Min.   :  0.0        Min.   : 0.00
##  1st Qu.: 5.828   1st Qu.:116.0        1st Qu.: 8.34
##  Median : 9.966   Median :180.0        Median :10.05
##  Mean   :10.811   Mean   :187.5        Mean   :10.35
##  3rd Qu.:14.136   3rd Qu.:290.0        3rd Qu.:14.81
##  Max.   :63.853   Max.   :359.0        Max.   :16.10
##   cloud_cover pressure_millibars daily_summary
##  Min.   :0    Min.   :   0       Length:96453
##  1st Qu.:0    1st Qu.:1012       Class :character
##  Median :0    Median :1016       Mode  :character
##  Mean   :0    Mean   :1003
##  3rd Qu.:0    3rd Qu.:1021
##  Max.   :0    Max.   :1046
```

```
#no negatives except for temperature, which is good, other values seem earthly.

summary_values <- weather_df %>%
  select_if(is.numeric) %>%
  summarize(across(
 everything(),
          list(
        count = ~sum(!is.na(.)),
        mean  = ~mean(., na.rm = TRUE),
        sd    = ~sd(., na.rm = TRUE),
        max   = ~max(., na.rm = TRUE),
        min   = ~min(., na.rm = TRUE)
      ),
      .names = "{.col}_{.fn}"
  )
  )


summary_values <- pivot_longer(summary_values,cols = colnames(summary_values),
    names_to = c("column")
    )
summary_values
```

```
## # A tibble: 40 x 2
##    column                          value
##    <chr>                           <dbl>
##  1 temperature_c_count          96453
##  2 temperature_c_mean              11.9
##  3 temperature_c_sd                 9.55
##  4 temperature_c_max               39.9
##  5 temperature_c_min              -21.8
##  6 apparent_temperature_c_count 96453
##  7 apparent_temperature_c_mean     10.9
##  8 apparent_temperature_c_sd       10.7
##  9 apparent_temperature_c_max      39.3
## 10 apparent_temperature_c_min     -27.7
## # i 30 more rows
```

```r
missing_values <- weather_df %>%
  select_if(is.numeric) %>%
  summarize(
    across(
      everything(),
      ~sum(is.na(.)),
      .names = "{.col}_n_NAs"
    )
  )

missing_values # no missing values
```

```
##   temperature_c_n_NAs apparent_temperature_c_n_NAs
## 1                   0                            0
##   humidity_n_NAs wind_speed_km_h_n_NAs
## 1              0                     0
##   wind_bearing_degrees_n_NAs visibility_km_n_NAs
## 1                          0                   0
##   cloud_cover_n_NAs pressure_millibars_n_NAs
## 1                 0                       0
```

```r
weather_df$formatted_date <- as_datetime(weather_df$formatted_date)
weather_df$hour <- hour(weather_df$formatted_date)
weather_df$day <- day(weather_df$formatted_date) # day of month
weather_df$month <- month(weather_df$formatted_date)
weather_df$year <- year(weather_df$formatted_date)

head(weather_df)
```

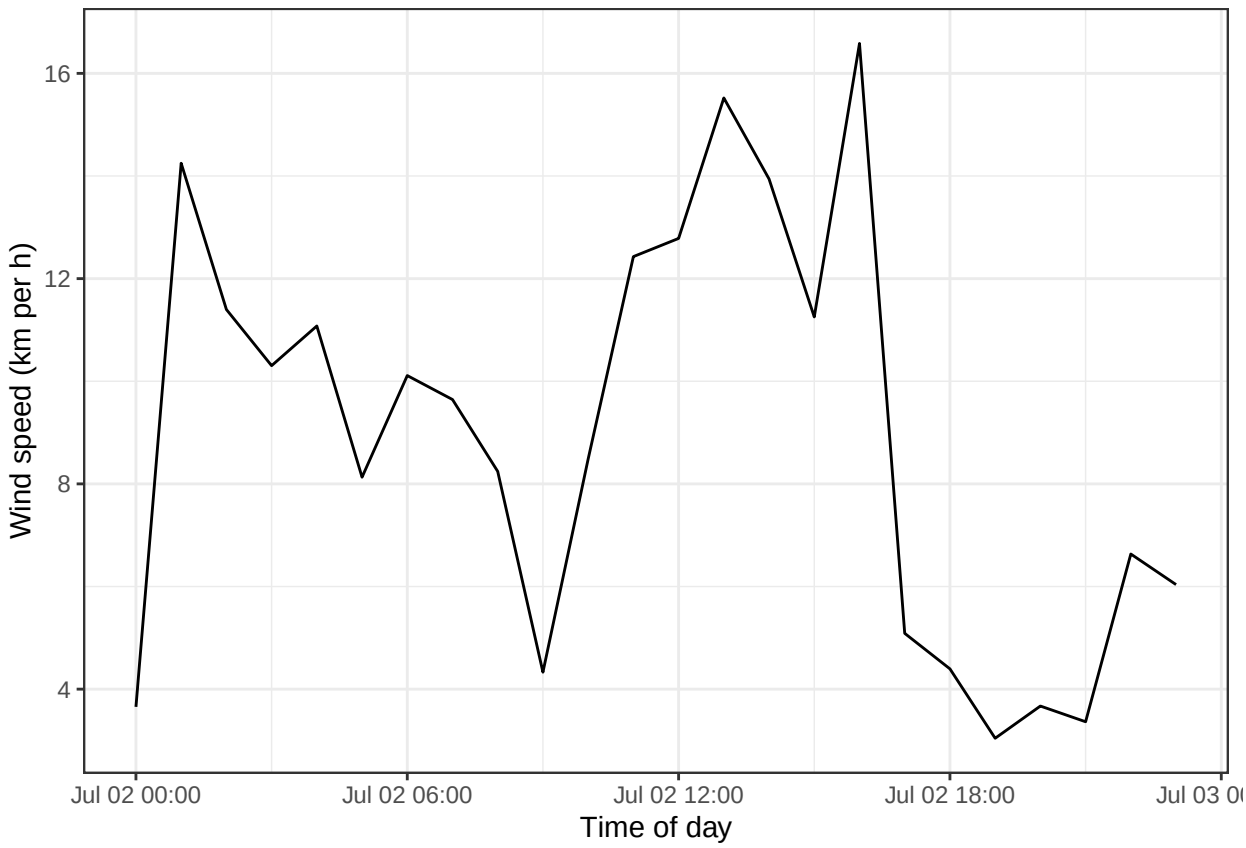
```
##        formatted_date       summary precip_type
## 1 2006-03-31 22:00:00 Partly Cloudy        rain
## 2 2006-03-31 23:00:00 Partly Cloudy        rain
## 3 2006-04-01 00:00:00 Mostly Cloudy        rain
## 4 2006-04-01 01:00:00 Partly Cloudy        rain
## 5 2006-04-01 02:00:00 Mostly Cloudy        rain
## 6 2006-04-01 03:00:00 Partly Cloudy        rain
##   temperature_c apparent_temperature_c humidity
## 1      9.472222               7.388889     0.89
## 2      9.355556               7.227778     0.86
## 3      9.377778               9.377778     0.89
## 4      8.288889               5.944444     0.83
## 5      8.755556               6.977778     0.83
## 6      9.222222               7.111111     0.85
##   wind_speed_km_h wind_bearing_degrees visibility_km
## 1         14.1197                  251       15.8263
## 2         14.2646                  259       15.8263
## 3          3.9284                  204       14.9569
## 4         14.1036                  269       15.8263
## 5         11.0446                  259       15.8263
## 6         13.9587                  258       14.9569
##   cloud_cover pressure_millibars
## 1           0            1015.13
## 2           0            1015.63
## 3           0            1015.94
```

```
## 4           0            1016.41
## 5           0            1016.51
## 6           0            1016.66
##                        daily_summary hour day month year
## 1 Partly cloudy throughout the day.   22  31     3 2006
## 2 Partly cloudy throughout the day.   23  31     3 2006
## 3 Partly cloudy throughout the day.    0   1     4 2006
## 4 Partly cloudy throughout the day.    1   1     4 2006
## 5 Partly cloudy throughout the day.    2   1     4 2006
## 6 Partly cloudy throughout the day.    3   1     4 2006
```

Now create multiple plots to continue exploring the dataset:

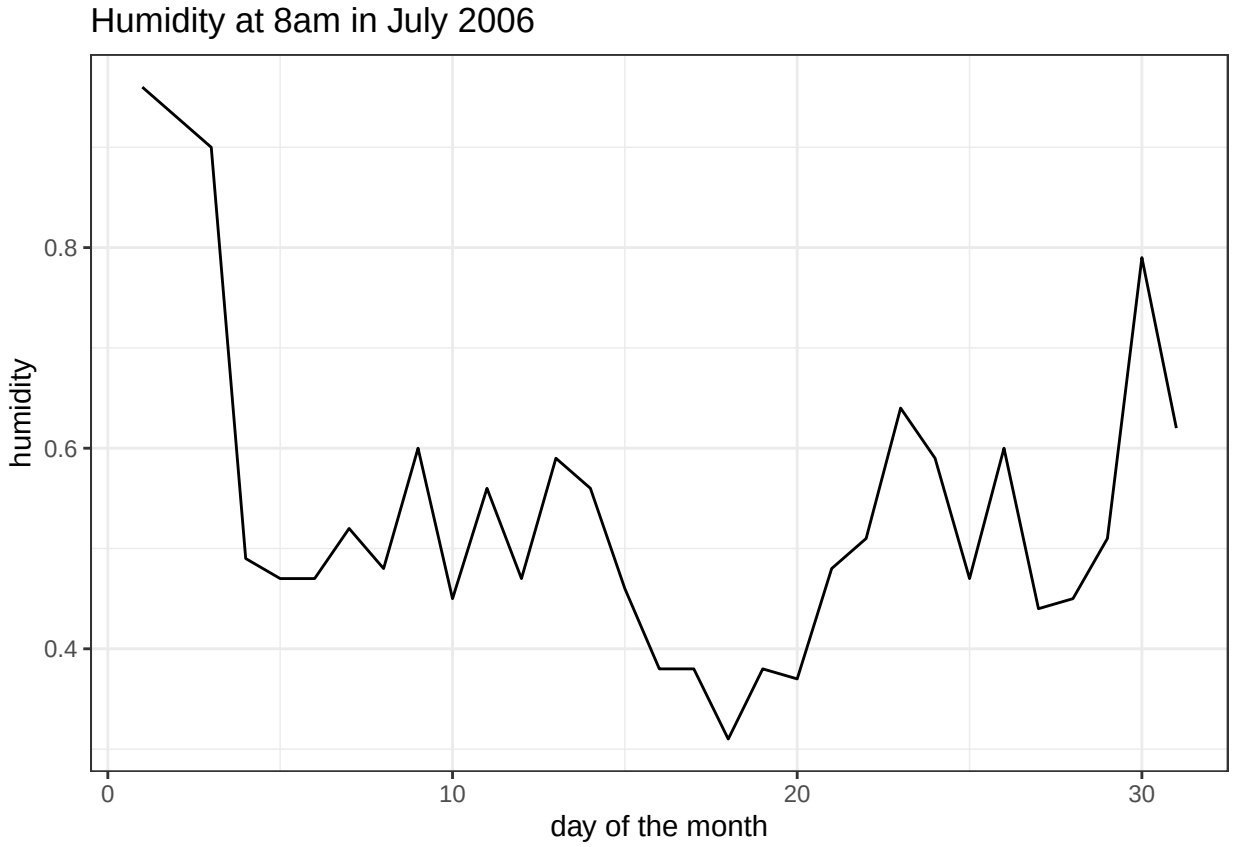
1. Plot a time-series graph of wind speed.
2. Create a second time-series graph of air temperature and humidity using two different axes.
3. Plot daily average temperature by calendar date; for example, average all January 1 observations, all January 2 observations, and so on.
4. Make the plots polished and readable.

```r
# wind vs time
weather_df_oneDay <- weather_df %>%
  filter(month==7, day == 2, year == 2006)
ggplot()+
  geom_line(data=weather_df_oneDay, aes(x=formatted_date, y=wind_speed_km_h))+
  ylab("Wind speed (km per h)")+
  xlab("Time of day")+ # i think this is UTC time
  theme_bw()
```



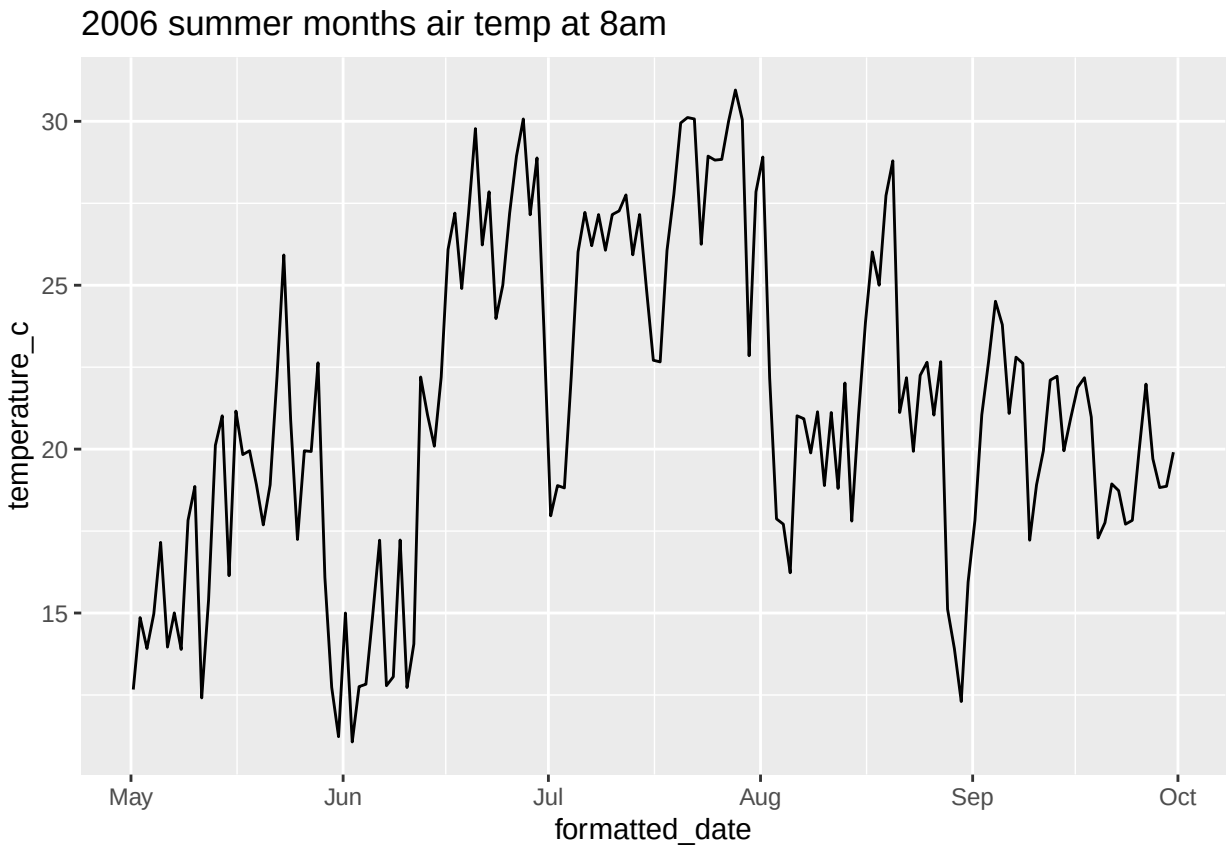
```
#humidity of july month
weather_df_oneMonth <- weather_df %>%
  filter(month==7, year == 2006, hour == 8 ) %>%
  ungroup()

ggplot(data=weather_df_oneMonth)+
  geom_line(aes(x=day, y=humidity))+
  ylab("humidity")+
  xlab("day of the month")+
labs(title="Humidity at 8am in July 2006")+
  theme_bw()
```




```
#airtemp of 2006 summer months at 8am
weather_df_coupleMonths <- weather_df %>%
  filter(month %in% c(5:9), year==2006, hour == 8 ) %>%
  ungroup()

ggplot(data=weather_df_coupleMonths)+
  geom_line(aes(x=formatted_date, y=temperature_c))+
  labs(title="2006 summer months air temp at 8am")
```



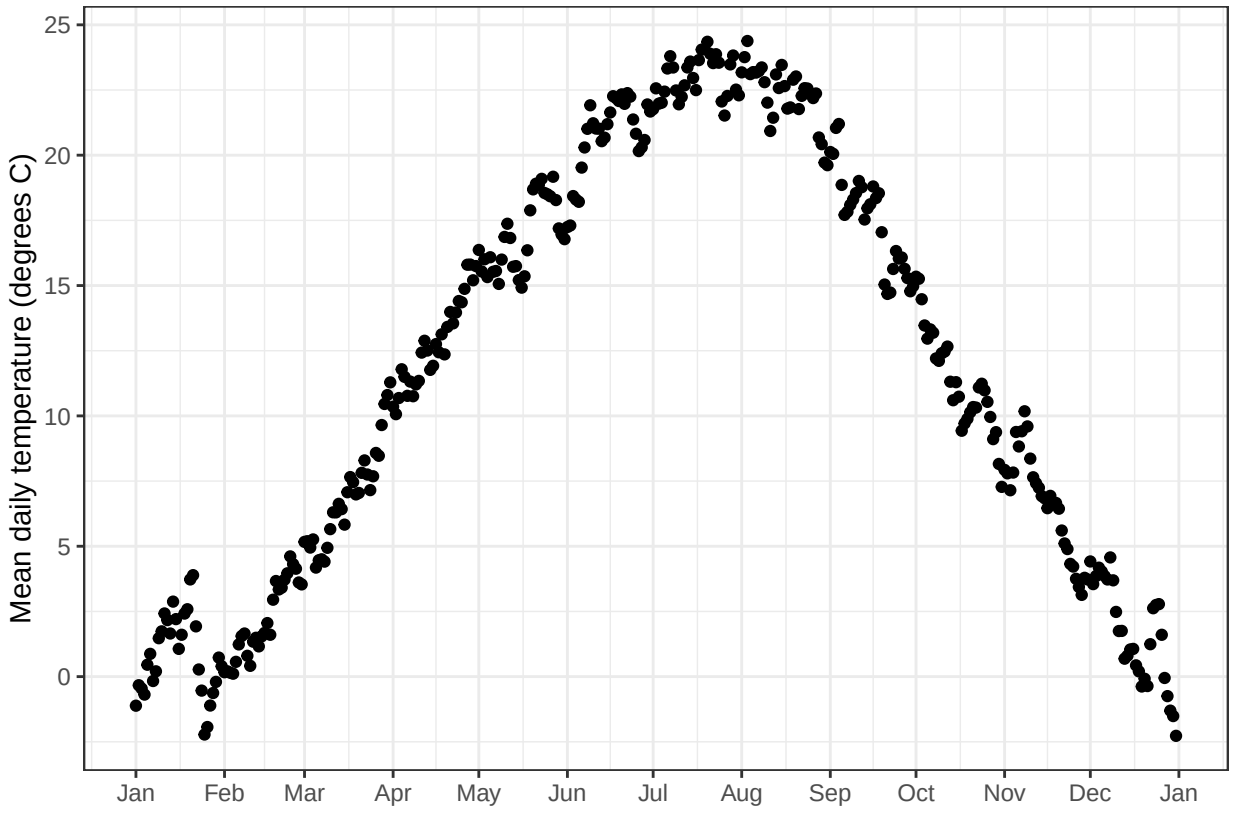

```
  ylab("Air temperature (degrees C)")+
  theme_bw()
```

```
## NULL
```

```
weather_df_daily_mean_temp <- weather_df %>%
  select(day, month, temperature_c) %>%
  group_by(month, day)%>%
  summarize(mean_temp = mean(temperature_c, na.rm=TRUE))

weather_df_daily_mean_temp<- weather_df_daily_mean_temp %>%
  mutate(
    fake_date = as.Date(sprintf("%04d-%02d-%02d", 2026, month, day))
  )

ggplot(data= weather_df_daily_mean_temp)+
  geom_point(aes(x=fake_date, y=mean_temp))+
  ylab("Mean daily temperature (degrees C)")+
  xlab("")+
  scale_x_date(  date_breaks = "1 month",
  date_labels = "%b")+
  theme_bw()
```



Rubric

Component	Points
Warm up	5
Load and copy data	2
Clean column names	3
Clean dataset	6
Plots (3 points each)	9
Total	**25**